

Sediment Core Analysis

You need to follow these first steps to get your data

Use this notebook to start your analysis.

```
In [23]: %matplotlib ipympl
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
from matplotlib.backends.backend_pdf import PdfPages
np.set_printoptions(legacy='1.25') #removes the funny printing
```

Read in Landes Core Data

```
In [12]: df=pd.read_excel('Landes_STOTEN.xlsx',sheet_name='Core_XRF')
```

```
In [13]: df.columns
```

```
Out[13]: Index(['core_id', 'sample_id_long', 'depth_cm_range', 'fine_coarse',
               'sampling_neighborhood', 'XRF_Pb_ppm', 'XRF_date', 'Nr_Runs', 'Pb',
               'Pb_sd', 'Pb_ins_e', 'Sn', 'Sn_sd', 'Sn_ins_e', 'Zn', 'Zn_sd',
               'Zn_ins_e', 'Cd', 'Cd_sd', 'Cd_ins_e', 'Unit'],
              dtype='object')
```

drop columns we don't need which are the errors

```
In [14]: df.drop(columns=['XRF_Pb_ppm', 'XRF_date', 'Nr_Runs', 'Pb_sd', 'Pb_ins_e', 'Sn_sd', 'Sn_ins_e',
                          'Zn_ins_e', 'Cd_sd', 'Cd_ins_e'],inplace=True)
```

check the columns we have left

```
In [15]: df.columns
```

```
Out[15]: Index(['core_id', 'sample_id_long', 'depth_cm_range', 'fine_coarse',
               'sampling_neighborhood', 'Pb', 'Sn', 'Zn', 'Cd', 'Unit'],
              dtype='object')
```

Delete unwanted data in depth column and replace with NaN

```
In [16]: df = df.replace('PL_UG', np.nan)
df = df.replace('PL_UG2', np.nan)
df = df.replace('PL_UG3', np.nan)
df = df.replace("PL_UG (can't find) ", np.nan)

df = df.replace('NA', np.nan)
```

```
In [17]: df
```

Out[17]:

	core_id	sample_id_long	depth_cm_range	fine_coarse	sampling_neighborhood	Pb
0	18NYHC001	18NYHC001_00_05_C	00_05	c	Greenpoint	728.333333
1	18NYHC001	18NYHC001_00_05_F	00_05	f	Greenpoint	1323.333333
2	18NYHC001	18NYHC001_05_10_C	05_10	c	Greenpoint	809.000000
3	18NYHC001	18NYHC001_05_10_F	05_10	f	Greenpoint	1524.000000
4	18NYHC001	18NYHC001_10_15_C	10_15	c	Greenpoint	485.666667
...	...	...	...	...	...	...
709	18UWHC022	18UWHC022_15_20_F	15_20	f	UWS	584.666667
710	18UWHC022	18UWHC022_20_26_C	20_26	c	UWS	939.000000
711	18UWHC022	18UWHC022_20_26_F	20_26	f	UWS	598.333333
712	18UWHC022	18UWHC022_PL_UG_C	NaN	c	UWS	NaN
713	18UWHC022	18UWHC022_PL_UG_F	NaN	f	UWS	NaN

714 rows × 10 columns

IMPORTANT We do not have weights for these cores. Franziska did not put it in the the data. We are going to assume all samples are 20.0 grams.

In [18]: `df['weight']=20.0`

Convert Depth Range to a mean and save to a new column

In [19]: `df['depth_cm']=df['depth_cm_range'].str.split('_',expand=True)\n .astype(float).mean(axis=1)`

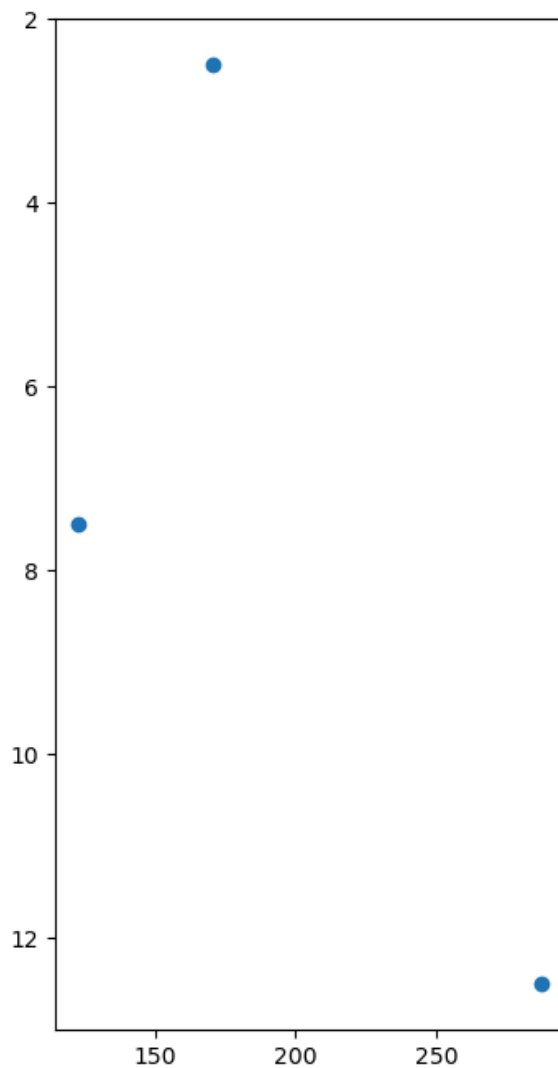
Just Save your core and the fine or coarse data to work with

In [20]: `core_id='18UWHC015'\n fine_coarse='f'\n\ndf_fine=df[(df['core_id']==core_id) & (df['fine_coarse']==fine_coarse)]`

Plot Lead

In [21]: `fig,ax=plt.subplots()\nfig.set_size_inches(4,8)\nax.scatter(df_fine.Pb,df_fine.depth_cm)\nax.invert_yaxis()`

Figure



In []:

Calculating Inventory

In the handout I gave the formula for calculating inventory but people are used to doing math in Pandas so here are some hints.

I can sum one column like lead

```
In [24]: df_fine['Pb'].sum()
```

```
Out[24]: 580.333333
```

I can sum a column and divide by a number

```
In [25]: df_fine['Pb'].sum()/1000
```

```
Out[25]: 0.580333333
```

I can multiply two columns. Here I will multiply depth and Pb

```
In [26]: df_fine['Pb']*df_fine['depth_cm']
```

```
Out[26]: 623      425.833333
        625      920.000003
        627     3591.666662
        629           NaN
        631           NaN
        633           NaN
        635           NaN
        637           NaN
        dtype: float64
```

I can multiply and then sum!

```
In [27]: (df_fine['Pb']*df_fine['depth_cm']).sum()
```

```
Out[27]: 4937.4999975
```

This is where you can use the function sum in two ways

```
In [28]: np.sum(df_fine['Pb']*df_fine['depth_cm'])
```

```
Out[28]: 4937.4999975
```

```
In [29]: (df_fine['Pb']*df_fine['depth_cm']).sum()
```

```
Out[29]: 4937.4999975
```

You have everything you need to know to calculate an inventory. It is easier than you think once you get your units in order!

The central park inventories average around 57 g/m2 from table 1.

A fun trick. You can make a second axis called a twin and plot on that.

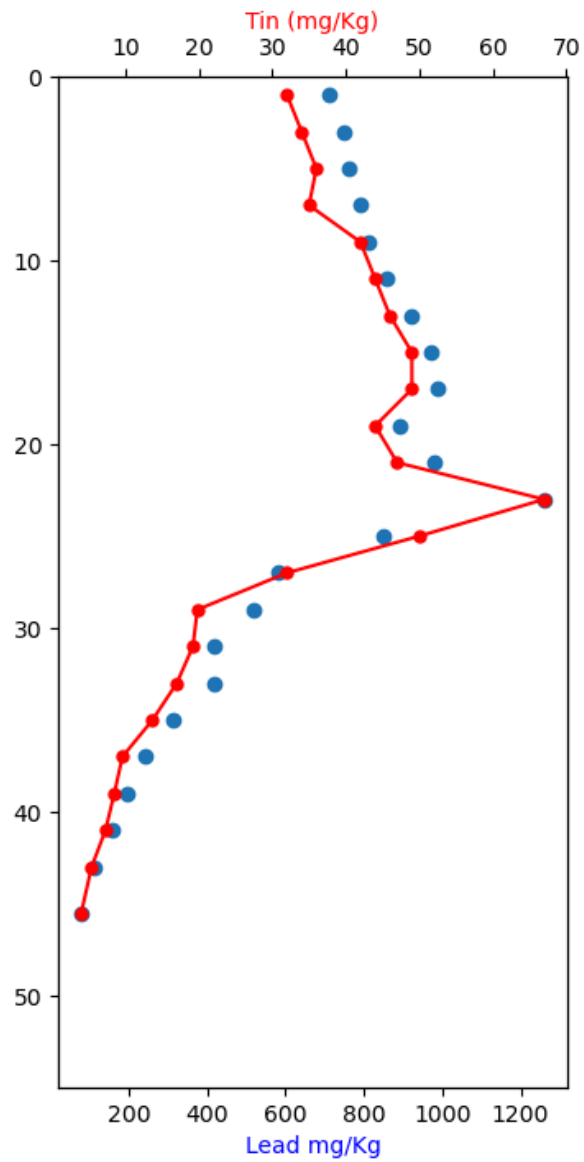
```
In [31]: df=pd.read_excel('ChillrudTable2.xlsx')

fig,ax=plt.subplots()
fig.set_size_inches(4,8)
ax.scatter(df.Pb,df.Depth_cm)
ax.set_ylim([55,0])
ax.margins(.05)
ax.set_xlabel('Lead mg/Kg',color='blue')

# This gives you a second axis with a nice plot in just a couple lines.
ax2=ax.twinx()
ax2.plot(df.Sn,df.Depth_cm,'.r-',markersize=10)
ax2.set_xlabel('Tin (mg/Kg)',color='r')
```

```
Out[31]: Text(0.5, 0, 'Tin (mg/Kg)')
```

Figure



Ignore after this.

My notes for assigning cores

```
In [21]: df_cores=pd.read_excel('Landes_STOTEN.xlsx',sheet_name='Core_XRF')
```

```
In [22]: df_cores['core_id'].value_counts()
```

```
Out[22]: core_id
18NYHC042    24
18UWHC004    24
18NYHC038    23
18UWHC009    21
18UWHC014    20
18UWHC018    18
18NYHC043    18
18UWHC012    17
18NYHC020    17
18NYHC044    16
18NYHC041    16
18NYHC029    16
18UWHC011    16
18UWHC019    16
18UWHC015    16
18NYHC021    15
18NYHC011    15
18NYHC045    15
18UWHC010    14
18NYHC013    14
18NYHC034    14
18NYHC008    14
18NYHC025    14
18NYHC019    13
18UWHC001    12
18UWHC013    12
18UWHC007    12
18UWHC006    12
18UWHC017    12
18NYHC001    12
18UWHC022    12
18NYHC012    12
18NYHC057    12
18NYHC003    12
18NYHC033    12
18NYHC031    12
18NYHC005    12
18NYHC026    12
18NYHC006    12
18NYHC015    12
18NYHC009    12
18UWHC020    10
18UWHC002    10
18NYHC014    10
18UWHC003    10
18UWHC008    10
18NYHC024    10
18UWHC016     8
18NYHC028     8
Name: count, dtype: int64
```

```
In [ ]:
```

Cores to use

18NYHC042

18UWHC004

18NYHC038

18UWHC009

18UWHC014

18UWHC018

18NYHC043

18UWHC012

18NYHC020

18NYHC044

18NYHC041

18NYHC029

18UWHC019